

Web Cache Deception — Exploiting origin server normalization for web cache deception

Plataforma: PortSwigger Web Security Academy

Ferramenta: Burp Suite

Categoria: Web Security / Cache Poisoning & Cache Deception

Escopo: Laboratório controlado

1. Objetivo

Identificar uma discrepância na codificação de delimitadores na forma como são decodificados pelo servidor de origem e cache interpretam delimitadores presentes no caminho da URL e utilizar essa diferença para demonstrar um cenário de **Web Cache Deception**.

O objetivo final do laboratório foi obter, em ambiente controlado, a chave de API associada ao usuário **Carlos**.

2. Descrição da vulnerabilidade

Web Cache Deception ocorre quando um atacante consegue induzir um mecanismo de cache, como um proxy reverso ou CDN, a armazenar uma resposta dinâmica ou sensível como se fosse um recurso estático.

O problema surge quando diferentes componentes da infraestrutura interpretam o mesmo caminho de URL de maneiras distintas:

- O servidor de origem interpreta a requisição como um endpoint dinâmico;
- O mecanismo de cache considera o caminho como um recurso estático e potencialmente cacheável;
- A resposta dinâmica pode, então, ser armazenada e posteriormente disponibilizada por uma URL aparentemente pública.

3. Conceitos observados

Path Confusion

O servidor de origem e o mecanismo de cache interpretam o mesmo caminho de URL de formas diferentes.

Diretórios estáticos

Habitualmente servidores web armazenam recursos estáticos em diretórios específicos. As regras de cachê geralmente têm alvo desses diretórios combinando prefixos de caminho de

URL específicos, como `/static`, `/resources` ou `/images`. Essas regras podem ser vulneráveis a fraudes de cachê web.

Discrepância na codificação de delimitadores

A fim de garantir que caracteres que têm um significado especial sejam interpretados como dados, eles geralmente são codificados. No entanto, alguns analisadores decodificam determinados caracteres antes de processar a URL. Diferenças em quais caracteres delimitadores são codificados pelo cachê e pelo servidor de origem podem resultar em discrepâncias na forma como eles interpretam a URL, ele poderá ser tratado como um delimitador, truncando o caminho da URL.

Requisição forjada

A técnica consiste em combinar um endpoint sensível com um caminho que aparenta representar um recurso estático.

4. Impacto potencial

A exploração desse tipo de falha pode resultar em:

- Exposição de dados sensíveis;
- Vazamento de chaves de API;
- Exposição de tokens ou credenciais;
- Acesso não autorizado a informações de outros usuários;
- Persistência temporária dos dados enquanto a resposta permanecer armazenada no cache.

5. Ambiente e acessos

O laboratório disponibiliza:

- Uma conta comum para análise do comportamento da aplicação;
- A página inicial da aplicação;
- O endpoint `My Account`;
- Uma lista de possíveis caracteres delimitadores para teste.

Credenciais do laboratório: `wiener:peter`

As credenciais acima pertencem exclusivamente ao ambiente controlado fornecido pelo laboratório.

6. Metodologia

6.1 Identificação do endpoint dinâmico

Foi identificado o endpoint **My Account**, responsável por retornar informações específicas do usuário autenticado, incluindo uma chave de API.

6.2 Análise das regras de recursos estáticos

Foi identificado o diretório utilizado pela aplicação para recursos aparentemente estáticos:

- **/resources**

O objetivo foi identificar padrões que poderiam levar o mecanismo de cache a classificar uma URL como cacheável.

6.3 Teste de delimitadores

O **Burp Suite Intruder** foi utilizado para automatizar o teste dos caracteres fornecidos pelo laboratório.

Os payloads foram inseridos em uma posição controlada entre o diretório que armazena recursos estáticos e o caminho do endpoint dinâmico.

6.4 Identificação do delimitador

O caractere (..) para voltar uma pasta mais o caractere (/) codificado para (%2f) apresentou discrepância relevante.

A análise demonstrou que o comportamento vulnerável dependia da combinação entre:

1. Um caminho com aparência de diretório para recurso estático.
2. Um endpoint dinâmico;
3. O delimitador codificado identificado;

7. Construção da requisição

A estrutura conceitual utilizada foi:

`resources/..%2f/my-account`

Nessa estrutura, os componentes da aplicação passaram a interpretar o caminho de maneiras diferentes.

O servidor de origem tratou a parte anterior ao delimitador de acordo com o mapeamento do endpoint dinâmico, enquanto o mecanismo de cache considerou o caminho completo e sua aparência de recurso estático.

8. Validação da exploração

A discrepância identificada permitiu induzir o armazenamento de uma resposta dinâmica no mecanismo de cache.

Após a validação do comportamento, foi possível acessar a chave de API associada ao usuário-alvo, concluindo o objetivo do laboratório.

9. Resultado

Status: Concluído com sucesso.

O laboratório demonstrou que uma discrepância na interpretação de delimitadores de caminho pode permitir que conteúdo dinâmico e sensível seja armazenado indevidamente por um mecanismo de cache.

A combinação de (..) com %2f, associada a um caminho que aparentava referenciar um recurso estático, foi o elemento relevante para reproduzir o comportamento vulnerável.

10. Conclusão

O exercício demonstrou a importância de analisar a forma como diferentes componentes de uma aplicação interpretam caminhos de URL.

Mesmo quando o servidor de origem processa corretamente um endpoint dinâmico, regras inadequadas no mecanismo de cache podem resultar na exposição de conteúdo sensível.

O Burp Suite foi utilizado tanto para análise manual quanto para automatização dos testes de delimitadores, permitindo identificar a discrepância e validar o comportamento observado.

11. Evidências

As evidências visuais da execução, incluindo capturas de tela das requisições, testes realizados no Burp Suite Intruder e validação do comportamento de cache, serão disponibilizadas separadamente em um arquivo de evidências.

12. Referência do laboratório

PortSwigger Web Security Academy — Web Cache Deception